TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN
A DISTANCIA



Metodología de Sistemas II

Trabajo Práctico
Flujo de trabajo colaborativo

Farias, Gustavo
Matrícula: 101662
Comisión: M2025-13
Repositorio GitHub: [Lucenear/Tecnicatura_Programacion_UTN]

Parte 1: Estrategias de Branching (Modelos de Ramificacion)

1- Analisis de Modelos:

Yo veo que GitFlow tiene una estructura mas jerarquica, con ramas de larga duracion, lo que aumenta la deuda de integracion y el riesgo de conflictos al final. En cambio, Trunk-based Development fomenta una comunicacion mas sincronizada y horizontal, con integraciones diarias. Esto reduce drasticamente la deuda de integracion y los conflictos, ya que los cambios son pequeños y constantes.

2- Caso de Estudio: Seleccion de Estrategia:

Para este escenario, yo recomendaria GitFlow. Al ser un sistema critico que requiere auditorias estrictas y lanzamientos espaciados, necesito un control rigido sobre las ramas de release y hotfix. GitFlow me da la estabilidad y el aislamiento necesarios para probar a fondo antes de fusionar a la rama principal, alineandose con la estructura jerarquica del equipo.

3-Feature Flags (Interruptores de Codigo):

Yo uso las Feature Flags para envolver el codigo nuevo en una condicion. Asi, puedo subir mi codigo incompleto a la rama principal todos los dias sin que el usuario final lo vea o lo active. Esto me permite integrar seguido sin romper la experiencia del usuario, apagando el interruptor hasta que la funcionalidad este lista y probada.

Parte 2: Procesos de Integracion Colaborativa

4- El Pull Request (PR) como Herramienta de Comunicacion:

En mi descripcion de PR, yo siempre incluyo el por que del cambio, un resumen de alto nivel de lo que se modifico, como probarlo paso a paso, y capturas de pantalla si aplica. Esto le da a mi compañero el contexto necesario para entender el cambio sin tener que leer cada linea de codigo de entrada.

5- La Cultura de la Code Review:

Para mi, la revision de codigo va mas alla de buscar errores; es una herramienta clave para transferir conocimiento entre compañeros y asegurar la calidad. Me ayuda a verificar que se cumplan principios como DRY y SOLID, mejorando el diseño general y asegurando que todos entendamos como funciona el sistema.

6- Estrategias de Merge y el Historial:

- Create a Merge Commit: Yo lo uso cuando quiero un historial veridico que muestre exactamente cuando y como se fusionaron las ramas, aunque el grafo se vuelva un poco desordenado.
- Squash and Merge: Lo prefiero cuando quiero un historial limpio, agrupando todos los commits de una funcionalidad en uno solo con un mensaje claro, ocultando los borradores intermedios.
- Rebase and Merge: Lo elijo para mantener un historial lineal y ordenado, como si todos los cambios se hubieran hecho uno detras del otro en la rama principal, sin crear commits de fusion extra.

Parte 3: Estandarizacion y Documentacion del Proyecto

7- Guias de Contribucion (CONTRIBUTING.md):

Si llega un nuevo integrante, yo espero que encuentre en el CONTRIBUTING.md las reglas de nombrado de ramas y commits, como configurar su entorno local, los pasos para correr las pruebas y el proceso exacto para abrir un PR. Esto le permite empezar a codificar rapido y siguiendo los estandares del equipo.

8- Architecture Decision Records (ADRs):

Yo valoro los ADRs porque documentan el por que de una decision tecnica, el contexto y las alternativas descartadas. Si un desarrollador se suma un año despues, este documento le evita adivinar o cometer los mismos errores, explicando claramente la razon detras de la arquitectura actual.

9- Plantillas de Trabajo (Templates):

El uso de plantillas para Issues y PRs me aporta orden y consistencia. Me aseguro de que nadie olvide informacion clave, como los pasos para reproducir un bug o la lista de verificacion antes de fusionar, agilizando la revision y la organizacion diaria del equipo.

Parte 4: Validacion de Estandares Locales

10- Git Hooks y la Primera Linea de Defensa:

Yo uso Git Hooks (como Husky) para ejecutar scripts automaticos antes de permitir un commit o un push. Esto me ayuda a validar el Linting y el formato del codigo en mi propio equipo local. Es crucial hacerlo localmente porque evita que codigo con errores de estilo o fallos basicos llegue al repositorio compartido, ahorrando tiempo de revision y manteniendo el estandar del equipo desde el origen.